

PROYECTO - MONITOR DE CORRIENTE CON EL INA236

Rodrigo Castillejos

CONTENTS

1	Introducción	3
1.1	¿Qué es un amplificador de detección de corriente?	3
1.2	Parámetros clave	3
1.3	Tipos de mediciones de corriente	3
2	Diseño de librería para el INA236	7
2.1	Headers	7
2.2	Desarrollando la estructura del header para el INA236	7
2.3	header	7
2.4	.c file	11

LIST OF FIGURES

Figure 1	Ejemplo de medición de corriente de lado alto (high-side).	4
Figure 2	Ejemplo de medición de corriente de lado bajo (low-side).	5
Figure 3	Ruta de la señal de detección de corriente	6

LIST OF TABLES

RESUMEN

Este documento presenta el diseño e implementación de un sistema de medición de corriente basado en el monitor de potencia, corriente y voltaje ultra preciso de 16 bits con interfaz I²C, el INA236 de Texas Instruments. Se abordará la creación de una librería en lenguaje C para la correcta comunicación entre el INA236 y una placa de desarrollo de STMicroelectronics (Nucleo 64) equipada con el microcontrolador STM32F303XX, aprovechando sus periféricos I²C y sus capacidades de procesamiento para aplicaciones de adquisición de datos. La librería permitirá no solo la lectura de corriente, voltaje y potencia, sino también la configuración dinámica del valor del resistor shunt, lo que facilita la adaptación del sistema a diferentes rangos de corriente sin modificar el hardware.

El diseño incluirá el cálculo del resistor de shunt además de la configuración de los registros internos del INA236 para ajustar las resoluciones y los rangos dinámicos. Finalmente, se desarrollará un diseño de PCB en Altium Designer. Este trabajo integra hardware, software y diseño electrónico para ofrecer una solución robusta en aplicaciones de monitoreo energético.

* Department of Biology, University of Examples, London, United Kingdom

¹ Department of Chemistry, University of Examples, London, United Kingdom

1 INTRODUCCIÓN

1.1 ¿Qué es un amplificador de detección de corriente?

Los amplificadores de detección de corriente (*current sense amplifiers*), también llamados monitores de derivación de corriente (*current shunt monitors*) son amplificadores diferenciales especializados con una red de ganancia resistiva adaptada con precisión con las siguientes características.

- Diseñados para monitorear el flujo de corriente midiendo la caída de voltaje a través de un elemento sensor, generalmente una resistencia de derivación o resistencia *shunt*.
- Suelen ser más fáciles de usar, más precisos y menos propensos al ruido.
- Soporta corrientes desde decenas de μA hasta cientos de A.
- Admite de forma nativa voltajes de modo común desde -16 a $+80$ V y con circuitería adicional hasta cientos de volts.

1.2 Parámetros clave

RANGO DE MODO COMÚN Esta especificación define el rango de voltaje de DC en la entrada del amplificador con respecto a tierra. Los amplificadores de detección de corriente se diseñan típicamente para soportar voltajes de modo común muy superiores a la tensión de alimentación del chip. Por ejemplo, el INA240 es capaz de soportar un voltaje de modo común de entre -4 V hasta 80 V mientras funciona con una fuente de alimentación de tan sólo 2.7 V.

VOLTAJE DE COMPENSACIÓN (OFFSET) Este es un error diferencial de DC en la entrada del amplificador. Historicamente, para reducir el impacto de los amplificadores con desajustes (*offset*) altos, se utilizaban resistencias *shunt* de mayor valor para aumentar la caída de tensión medida. Hoy en día, se ofrecen amplificadores de detección de corriente con desfases de tan sólo $10\ \mu\text{V}$, lo que permite mediciones de mayor precisión a bajas corrientes y el uso de resistencias *shunt* de menor valor para mejorar la eficiencia del sistema.

GANANCIA Los amplificadores de detección de corriente incorporan diversas opciones de ganancia que ofrecen un rendimiento robusto frente a variaciones de temperatura y proceso, gracias a la integración de una red de ganancia resistiva de ajuste preciso. Las opciones de ganancia para amplificadores de ganancia fija varían de $0.125\ \text{V/V}$ a $1000\ \text{V/V}$ con errores de ganancia de tan sólo $0.01\ \%$.

ESTABILIDAD DE TEMPERATURA Los amplificadores de detección de corriente integran el amplificador con todas las resistencias de ajuste de ganancia, lo que permite una desviación de temperatura pequeña y uniforme. Esto permite mediciones de corrientes robustas en todo el rango de temperatura especificado. La estabilidad de temperatura lograda es una de las principales ventajas de los amplificadores de detección de corriente sobre las implementaciones discretas.

1.3 Tipos de mediciones de corriente

1.3.1 Medición en el lado alto (*high-side*)

Esta técnica permite monitorear la corriente colocando el elemento sensor entre lado positivo del voltaje de bus y la carga.

Ventajas

- Capaz de detectar cortocircuito a tierra en la carga.
- La corriente se monitorea directamente desde la fuente.
- Alta inmunidad a perturbaciones con respecto a tierra.

Retos

- El alto voltaje de bus limita la disponibilidad de dispositivos con alto voltaje de entrada de modo común.

Ventajas sobre circuitos de detección de corriente discretos

- Las resistencias de ganancia integradas proporcionan una excelente adaptación para permitir un mayor rendimiento.
- Reducción en los requisitos de espacio en la placa.
- La arquitectura de entrada única permite que el voltaje de modo común supere en gran medida el voltaje de alimentación del circuito.

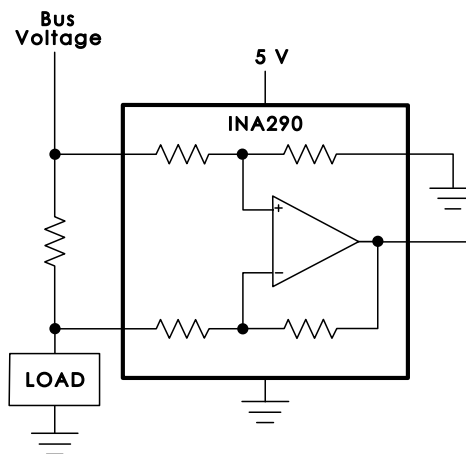


Figure 1: Medición de corriente de lado alto.

1.3.2 Medición en el lado bajo

Esta técnica permite monitorear la corriente colocando el elemento sensor entre la carga y tierra.

Ventajas

- Simple de implementar y solución de bajo costo.
- Amplio rango de opciones disponibles.

Retos

- Difícil de detectar cortocircuitos a tierra en la carga.
- Perturbaciones en el sistema a tierra por la resistencia de derivación.

Ventajas sobre circuitos de detección de corriente discretos

- Las resistencias de ganancia integradas proporcionan una excelente adaptación para permitir un mayor rendimiento.

- Reducción en los requisitos de espacio en la placa.
- Detecta una verdadera medición diferencial a través de la resistencia *shunt*.
- Un V_{OFFSET} más bajo ahorra energía del sistema al permitir el uso de resistencias *shunt* de menor valor para lograr el mismo nivel de error.

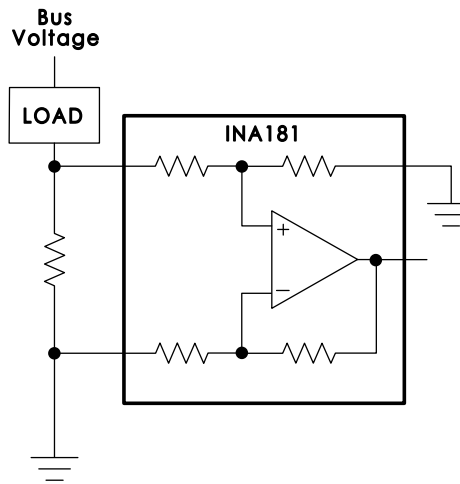


Figure 2: Medición de corriente de lado bajo.

Las mediciones de corriente se utilizan en sistemas electrónicos para proporcionar retroalimentación que verifique que el funcionamiento se encuentre dentro de los márgenes aceptables y para detectar posibles fallos. El análisis de nivel de corriente de un sistema permite diagnosticar modos de funcionamiento imprevistos o inesperados, lo que permite realizar ajustes para mejorar la fiabilidad o proteger los componentes del sistema contra daños.

La corriente es una señal difícil de medir directamente. Sin embargo, existen varios métodos de medición que permiten medir el efecto del flujo de corriente. La corriente que pasa por un cable produce un campo magnético que puede detectarse mediante sensores magnéticos (por ejemplo, de efecto Hall y de compuerta de flujo). La corriente también puede medirse midiendo el voltaje que se desarrolla en una resistencia al pasar la corriente. Este tipo de resistencia se denomina resistencia de detección de corriente (*current sensing*) o resistencia de derivación (*shunt*).

Para rangos de corriente de hasta 100 amperios en rangos de tensión inferiores a 100 voltios, se suele preferir la medición de corriente con resistencias en derivación. Este método suele proporcionar una medición físicamente más pequeña, más precisa y con mayor estabilidad térmica en comparación con una solución magnética.

Para evaluar y analizar la información actual del sistema, es necesario digitalizarla y enviarla al controlador. Existen diversos métodos para medir y convertir la señal generada a través de la resistencia *shunt*. El método más común consiste en utilizar un front-end analógico para convertir la señal diferencial de la resistencia de detección de corriente en una señal unipolar. Esta señal unipolar se conecta a un convertidor analógico-digital (ADC) conectado a un microcontrolador. La Figura 1 ilustra la cadena de señales de detección de corriente.

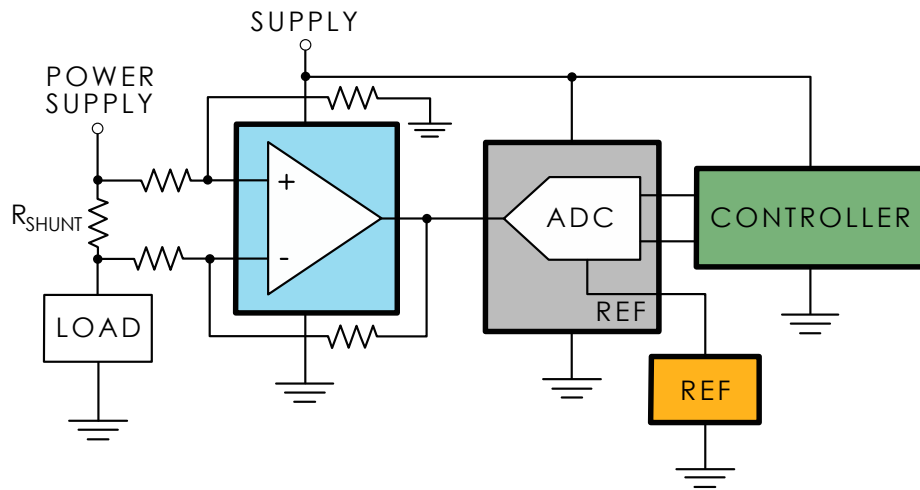


Figure 3: Ruta de la señal de detección de corriente.

2 DISEÑO DE LIBRERÍA PARA EL INA236

Uno de los beneficios de crear definiciones de clases es que, al empaquetarse correctamente, otros programadores pueden reutilizarlas.

2.1 Headers

Los archivos escritos en C o C++ normalmente consisten de un archivo `.c` o `.cpp`, también conocidos como **archivo de código fuente** (source-code file) que contienen la definición de la función `main`, así como definiciones de *clases* y otras funciones para la ejecución de un determinado programa. Al crear un programa orientado a objetos, se suele definir un código fuente reutilizable (como una clase) en un archivo que, por convención, tiene la extensión `.h` conocido como encabezado (*header*). Los programas usan las directivas de preprocesador `#include` para incluir encabezados y tomar ventaja de componentes de software reutilizables.

2.2 Desarrollando la estructura del header para el INA236

La directiva `#ifndef` se utiliza para comprobar si no se ha definido una macro con anterioridad. Si el identificador especificado no está definido como una macro, las líneas de código inmediatamente después de la condición se pasan al compilador. Por lo tanto, para evitar incluir múltiples inclusiones del mismo header dentro de la unidad de compilación, incluya las siguientes definiciones. Estas serán las primeras líneas de cada encabezado.

```
1 #ifndef _INA236_H_
2 #define _INA236_H_
3
4 // Declarations go here
5
6 #endif
```

La directiva `#endif` se usa para marcar el final del bloque de código que ha sido incluido condicionalmente por la directiva `#ifndef`. Para el especificador, incluya un nombre acorde con el sensor, componente o módulo que esté desarrollando, y procure colocarlo en mayúsculas.

Ahora definimos diferentes constantes y macros

2.3 header

```
1 /**
2 *****
3 * @file      : INA236.h
4 * @brief     : INA236 Library
5 *****
6
7 The INA236 is a 16-bit digital current monitor with an I2C interface
   that is compliant with a wide range of digital bus voltage such
   as 1.2V, 1.8V, 3.3V, and 5.0V. The device monitors the voltage
   across an external sense resistor and reports values for current
   , bus voltage, and power
8
9 @details
10
11 - High-side or low-side current sensing
12 - Operates from a 1.7V to 5.5V power supply
13 - Reports current, voltage and power
14 - Programmable full scale range: 20mV / 80mV
```

```

15 - 16-bit ADC resolution
16 - 0.1% gain error (maximum)
17 - 5uV offset (maximum)
18 - Low input bias current: 10nA (maximum)
19 - Configurable averaging options
20 - General call addressing allows conversion synchronization among
    devices
21 - Alert limits for over and under current events
22 - 1.2V compliant I2C bus, SMBus interface
23 - Two device address options with a 4-pin selectable address
24 - Operating temperature: -40 C y +125 C
25
26 * @example
27 * INA236_CALIBRATION_REGISTER is calculated based on the next
    equation
28 *  $SHUNT\_CAL = 0.00512 / (Current\_LSB \times R_{shunt})$ 
    .....(1)
29
30 Where:
31 * 0.00512 is an internal fixed value used to ensure scaling is
    maintained properly
32 * Current_LSB is a selected value for the current step size in A.
    Must be greater than or equal to CURRENT_LSB (minimum), but less
    than 8 x CURRENT_LSB (minimum) to reduce resolution loss.
33 * The SHUNT_CAL must be divided by 4 for ADCRANGE = 1
34
35 *  $CURRENT\_LSB \text{ (minimum)} = \text{Maximum Expected Current} /$ 
     $2^{15}$ .....(2)
36
37 * After programming the SHUNT_CAL register with the calculated value
    , the measured current in amperes can be read from the CURRENT
    register. Use equation (3) to calculate the final value scaled
    by the CURRENT_LSB:
38
39 *  $Current[A] = CURRENT\_LSB \times CURRENT$ 
    .....(3)
40
41 Where:
42 * CURRENT is the value read from the CURRENT register
43
44 @Design Parameters
45 - Power Supply Voltage (Vs) = 3.3V
46 - Bus Supply rail (VCM) = 12V
47 - Average Current = 6A
48 - OverCurrent fault threshold = 9A
49 - Maximum current monitored (Imax) = 10A
50 - ADC Range Selection (Vsense_max) = +-81.92mV
51
52 * 1. Select the Shunt Resistor
53 *
54 *  $R_{shunt} < V\_SENSE\_MAX / I\_MAX$ 
    .....(4)
55 *
56 *****
57 */
58
59
60 #ifndef INC_INA236_H_
61 #define INC_INA236_H_
62
63 #define INA236_DEFAULT_ADDRESS 0x40

```



```

64 #define INA236_TRIALS                    5
65 #define CHECK_BIT(var,pos) ((var) & (1<<(pos)))
66
67 // Registers
68 #define INA236_CONFIGURATION_REGISTER    0x00
69 #define INA236_SHUNT_VOLTAGE_REGISTER    0x01
70 #define INA236_BUS_VOLTAGE_REGISTER      0x02
71 #define INA236_POWER_REGISTER            0x03
72 #define INA236_CURRENT_REGISTER          0x04
73 #define INA236_CALIBRATION_REGISTER      0x05
74 #define INA236_MASK_ENABLE_REGISTER      0x06
75 #define INA236_ALERT_LIMIT_REGISTER      0x07 // Contains the value
        used to compare to the register selected in the Mask/Enable
        Register to determine if a limit has been exceeded
76 #define MANUFACTURER_ID_REGISTER        0x3E // Reset Value: 5449h
77 #define DEVICE_ID_REGISTER              0x3F // Reset Value: A080h
78
79 #define INA236_CONFIG_RESERVED          0x4000
80
81 // Mask_Enable Register
82 #define INA236_SHUNT_OVER_LIMIT          0x8000 // Setting this bit
        high configures the ALERT pin to be asserted if the shunt
        voltage conversion result exceeds the value programmed in the
        LIMIT register
83 #define INA236_SHUNT_UNDER_LIMIT        0x4000 // Setting this bit
        high configures the ALERT pin to be asserted if the shunt
        voltage conversion result is below the value programmed in the
        LIMIT register
84 #define INA236_BUS_OVER_LIMIT           0x2000 // Setting this bit
        high configures the ALERT pin to be asserted if the bus voltage
        conversion result exceeds the value programmed in the LIMIT
        Register.
85 #define INA236_BUS_UNDER_LIMIT          0x1000 // Setting this bit
        high configures the ALERT pin to be asserted if the bus voltage
        conversion result is below the value programmed in the LIMIT
        register.
86 #define INA236_POWER_OVER_LIMIT         0x0800 // Setting this bit
        high configures the ALERT pin to be asserted if the power result
        exceeds the value programmed in ther LIMIT register.
87 #define INA236_CONVERSION_READY         0x0400 // Setting this bit
        high configures the ALERT pin to be asserted when the
        Conversion Ready flag , bit 3, is asserted indicating that the
        device is ready for the next conversion.
88 #define INA236_MEM_ERROR                0x0020 // CRC or ECC error
89 #define INA236_ALERT_FUNCTION_FLAG      0x0010
90 #define INA236_CONVERSION_READY_FLAG    0x0008
91 #define INA236_MATH_OVER_FLOW           0x0004 // This bit is set to
        '1' if an arithmetic operation resulted in an overflow error.
        It indicates that current and power data may be invalid
92 #define INA236_ALERT_POLARITY           0x0002 // Alert polarity bit
        sets the alert pin polarity. 0b = Normal (Active-low open drain
        ). 1b = Inverted (active-high)
93 #define INA236_ALERT_LATCH_ENABLE        0x0001 // When the Alert
        latch Enable bit is set to Latch mode, the Alert pin and AFF bit
        remains active following a fault until this register flag has
        been read. This bit must be set to use the I2C Alert Response
        function
94
95 /**
96 * Enables the selection of the shunt full scale input across IN+ and
        IN-

```

```

97  */
98  typedef enum {
99      INA236_ADCRANGE_81DOT92_MILIVOLT    = 0x0000, // ADCRANGE = 0
        (+-81.92mV). 2.5uV/LSB
100     INA236_ADCRANGE_20DOT48_MILIVOLT    = 0x1000 // ADCRANGE = 1
        (+-20.48mV). 625nV/LSB
101 } AdcRange_t;
102
103 typedef enum {
104     INA236_1_SAMPLE                      = 0x0000,
105     INA236_4_SAMPLES                     = 0x0200,
106     INA236_16_SAMPLES                    = 0x0400,
107     INA236_64_SAMPLES                    = 0x0600,
108     INA236_128_SAMPLES                   = 0x0800,
109     INA236_256_SAMPLES                   = 0x0A00,
110     INA236_512_SAMPLES                   = 0x0C00,
111     INA236_1024_SAMPLES                   = 0x0E00
112 } Avg_t;
113
114 typedef enum {
115     INA236_VBUSCT_140_US                 = 0x0000,
116     INA236_VBUSCT_204_US                 = 0x0040,
117     INA236_VBUSCT_332_US                 = 0x0080,
118     INA236_VBUSCT_588_US                 = 0x00C0,
119     INA236_VBUSCT_1100_US                = 0x0100,
120     INA236_VBUSCT_2116_US                = 0x0140,
121     INA236_VBUSCT_4156_US                = 0x0180,
122     INA236_VBUSCT_8244_US                = 0x01C0
123 } VbusConversionTime_t;
124
125 typedef enum {
126     INA236_VSHCT_140_US                  = 0x0000,
127     INA236_VSHCT_204_US                  = 0x0008,
128     INA236_VSHCT_332_US                  = 0x0010,
129     INA236_VSHCT_588_US                  = 0x0018,
130     INA236_VSHCT_1100_US                 = 0x0020,
131     INA236_VSHCT_2116_US                 = 0x0028,
132     INA236_VSHCT_4156_US                 = 0x0030,
133     INA236_VSHCT_8244_US                 = 0x0038
134 } ShuntConversionTime_t;
135
136 typedef enum {
137     INA236_SHUTDOWN_MODE                  = 0x0000,
138     INA236_SHVOLT_TRIGGERED_ONE_SHOT     = 0x0001,
139     INA236_BUSVOLT_TRIGGERED_ONE_SHOT    = 0x0002,
140     INA236_SHUNTBUS_TRIGGERED_ONE_SHOT   = 0x0003,
141     INA236_SHUTDOWN                      = 0x0004,
142     INA236_CONTINUOUS_SHUNT_VOLTAGE       = 0x0005,
143     INA236_CONTINUOUS_BUS_VOLTAGE         = 0x0006,
144     INA236_CONTINUOUS_SHUNTBUS_VOLTAGE    = 0x0007
145 } Mode_t;
146
147 typedef enum {
148     SOL = 1, // Shunt Over Limit
149     SUL = 2, // Shunt Under Limit
150     BOL = 3, // Bus Over limit
151     BUL = 4, // Bus Under Limit
152     POL = 5, // Power Over Limit
153     CNVR = 6 // Conversion Ready
154 } AlertType_t;
155

```

```

156 typedef struct
157 {
158     I2C_HandleTypeDef *hi2c;
159     uint8_t          devAddress;
160     uint8_t          rangeAdc;
161     double           shuntResistor;
162     double           maximumCurrent;
163     double           shuntAdcRange;
164     double           currentLsbMin;
165     AlertType_t      alertType;
166 } Ina236_t;
167
168
169
170 void ina236writeRegister(Ina236_t *ina236,
171     uint8_t registerAddress,
172     uint16_t value);
173
174 uint16_t ina236readRegister(Ina236_t *ina236,
175     uint8_t registerAddress);
176
177 uint8_t ina236Init(Ina236_t *ina236,
178     I2C_HandleTypeDef *i2c,
179     uint8_t devAddress,
180     AdcRange_t adcRange,
181     Avg_t samples,
182     VbusConversionTime_t busConversionTime,
183     ShuntConversionTime_t shuntConversionTime,
184     Mode_t mode);
185
186
187 uint8_t ina236DefaulInit(Ina236_t *ina236, I2C_HandleTypeDef *i2c,
188     uint8_t devAddress);
189
190 void ina236SetCalibration(Ina236_t *ina236, double rShuntValue, int
191     maxCurrent);
192
193 uint8_t ina236SetMaskRegister(Ina236_t *ina236, AlertType_t
194     alertType);
195
196 void ina236ResetMaskRegister(Ina236_t *ina236);
197
198 void ina236SetCurrentAlertLimit(Ina236_t *ina236, double
199     currentLimit);
200
201 double ina236ReadShuntVoltage(Ina236_t *ina236);
202
203 double ina236ReadBusVoltage(Ina236_t *ina236);
204
205 double ina236ReadCurrent(Ina236_t *ina236);
206
207 double ina236ReadPower(Ina236_t *ina236);
208
209 _Bool ina236AlertFunctionFlag(Ina236_t *ina236);
210
211 _Bool ina236DataReady(Ina236_t *ina236);
212
213 #endif

```

2.4 .c file

```

1 #include "main.h"
2 #include "math.h"
3 #include "INA236.h"
4
5 /**
6  * @brief Write an amount of data in blocking mode to a specific
7  *         memory address
8  * @param ina219 Pointer to a I2C_HandleTypeDef structure that
9  *         contains
10 *             the configuration information for the specified I2C
11 *             .
12 * @param registerAddress Target device address: The device 7 bits
13 *         address value
14 *             in datasheet must be shifted to the left before calling
15 *             the interface
16 * @param value The data that must be written in the register
17 */
18 void ina236writeRegister(Ina236_t *ina236, uint8_t registerAddress,
19                          uint16_t value)
20 {
21     uint8_t address[2];
22     uint8_t isDeviceReady;
23
24     address[0] = (value >> 8) & 0xFF;
25     address[1] = (value >> 0) & 0xFF;
26
27     isDeviceReady = HAL_I2C_IsDeviceReady(ina236->hi2c, (ina236->
28         devAddress) << 1, INA236_TRIALS, HAL_MAX_DELAY);
29
30     if (isDeviceReady == HAL_OK)
31     {
32         HAL_I2C_Mem_Write(ina236->hi2c, (ina236->devAddress) << 1,
33             registerAddress, I2C_MEMADD_SIZE_8BIT, (uint8_t*)address,
34             I2C_MEMADD_SIZE_16BIT, HAL_MAX_DELAY);
35     }
36 }
37
38 /**
39  * @brief Read an amount of data in blocking mode from a specific
40  *         memory address
41  * @param hi2c Pointer to a I2C_HandleTypeDef structure that
42  *         contains
43 *             the configuration information for the specified I2C
44 *             .
45 * @param registerAddress Target device address: The device 7 bits
46 *         address value
47 * @retval 16-bit data read from register's device
48 */
49 uint16_t ina236readRegister(Ina236_t *ina236, uint8_t
50     registerAddress)
51 {
52     uint8_t registerResponse[2];
53     uint8_t isDeviceReady;
54
55     isDeviceReady = HAL_I2C_IsDeviceReady(ina236->hi2c, (ina236->
56         devAddress) << 1, INA236_TRIALS, HAL_MAX_DELAY);
57
58     if (isDeviceReady == HAL_OK)
59     {

```

```

45 HAL_I2C_Mem_Read(ina236->hi2c, (ina236->devAddress) << 1,
    registerAddress, I2C_MEMADD_SIZE_8BIT, registerResponse, sizeof(
    registerResponse), HAL_MAX_DELAY);
46 }
47
48 return ((registerResponse[0] << 8) | registerResponse[1]);
49 }
50
51 /**
52  * @brief Initializes the CONFIGURATION register's device with
    default values
53  * @param ina236 points to an object of the type Ina236_t
54  * @param hi2c Pointer to a I2C_HandleTypeDef structure that
    contains
55  *
    the configuration information for the specified I2C
    .
56  * @param devAddress Target device address: The device 7 bits
    address value
57  * @retval 16-bit data read from register's device
58  */
59 uint8_t ina236DefaulInit(Ina236_t *ina236, I2C_HandleTypeDef *i2c,
    uint8_t devAddress)
60 {
61     ina236->hi2c = i2c;
62     ina236->devAddress = devAddress;
63     ina236->shuntAdcRange = 2.5 * pow(10, -6);
64     uint16_t configValue;
65     uint8_t isDeviceReady;
66
67     configValue = INA236_CONFIG_RESERVED |
        INA236_ADCRANGE_81DOT92_MILIVOLT | INA236_1_SAMPLE |
        INA236_VBUSCT_1100_US | INA236_VSHCT_1100_US |
        INA236_CONTINUOUS_SHUNTBUS_VOLTAGE;
68     isDeviceReady = HAL_I2C_IsDeviceReady(i2c, devAddress << 1,
        INA236_TRIALS, HAL_MAX_DELAY);
69
70     if (isDeviceReady == HAL_OK)
71     {
72         //HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
73         ina236writeRegister(ina236, INA236_CONFIGURATION_REGISTER,
            configValue);
74         ina236ResetMaskRegister(ina236);
75         return 1;
76     }
77
78     return 0;
79 }
80
81 uint8_t ina236Init(Ina236_t *ina236, I2C_HandleTypeDef *i2c, uint8_t
    devAddress, AdcRange_t adcRange, Avg_t samples,
    VbusConversionTime_t busConversionTime, ShuntConversionTime_t
    shuntConversionTime, Mode_t mode)
82 {
83     ina236->hi2c = i2c;
84     ina236->devAddress = devAddress;
85     uint16_t configValue;
86     uint8_t isDeviceReady;
87
88     if (adcRange == INA236_ADCRANGE_81DOT92_MILIVOLT)
89     {
90         ina236->shuntAdcRange = 2.5 * pow(10, -6);

```

```

91     ina236->rangeAdc = 0;
92 }
93 else if (adcRange == INA236_ADCRANGE_20DOT48_MILIVOLT)
94 {
95     ina236->shuntAdcRange = 625 * pow(10, -9);
96     ina236->rangeAdc = 1;
97 }
98
99 configValue = INA236_CONFIG_RESERVED | adcRange | samples |
    busConversionTime | shuntConversionTime | mode;
100 isDeviceReady = HAL_I2C_IsDeviceReady(i2c, devAddress << 1,
    INA236_TRIALS, HAL_MAX_DELAY);
101
102 if (isDeviceReady == HAL_OK)
103 {
104     //HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
105     ina236writeRegister(ina236, INA236_CONFIGURATION_REGISTER,
        configValue);
106     return 1;
107 }
108
109 return 0;
110 }
111
112 /**
113  * @brief Initializes the CONFIGURATION register's device with
        default values
114  * @param ina236 points to an object of the type Ina236_t
115  * @param rShuntValue Shunt Resistor's Value
116  * @param maxCurrent Maximum current monitored
117  * @retval
118  */
119 void ina236SetCalibration(Ina236_t *ina236, double rShuntValue, int
    maxCurrent)
120 {
121     ina236->shuntResistor = rShuntValue;
122     ina236->maximumCurrent = maxCurrent;
123     double currentLsbMinimum;
124     uint16_t shuntCal;
125
126     currentLsbMinimum = maxCurrent / pow(2, 15);
127     if (maxCurrent < 3) currentLsbMinimum = 100.0 * pow(10, -6);
128     else if (maxCurrent > 3 && maxCurrent <= 6) currentLsbMinimum =
        200.0 * pow(10, -6);
129     else if (maxCurrent > 6 && maxCurrent <= 9) currentLsbMinimum =
        300.0 * pow(10, -6);
130     else if (maxCurrent > 9 && maxCurrent <= 13) currentLsbMinimum =
        500.0 * pow(10, -6);
131     else if (maxCurrent > 13 && maxCurrent <= 16) currentLsbMinimum =
        500.0 * pow(10, -6);
132     else if (maxCurrent > 16 && maxCurrent <= 19) currentLsbMinimum =
        600.0 * pow(10, -6);
133
134     ina236->currentLsbMin = currentLsbMinimum;
135
136     shuntCal = 0.00512 / (currentLsbMinimum * rShuntValue);
137
138     if (ina236->rangeAdc == 1)
139     {
140         shuntCal = shuntCal / 4;
141     }

```

```

142     ina236writeRegister(ina236, INA236_CALIBRATION_REGISTER, shuntCal);
143 }
144
145 uint8_t ina236SetMaskRegister(Ina236_t *ina236, AlertType_t
146     alertType)
147 {
148     ina236->alertType = alertType;
149     uint16_t value;
150
151     switch(alertType)
152     {
153         case 1:
154             value = INA236_SHUNT_OVER_LIMIT;
155             break;
156         case 2:
157             value = INA236_SHUNT_UNDER_LIMIT;
158             break;
159         case 3:
160             value = INA236_BUS_OVER_LIMIT;
161             break;
162         case 4:
163             value = INA236_BUS_UNDER_LIMIT;
164             break;
165         case 5:
166             value = INA236_POWER_OVER_LIMIT;
167             break;
168         case 6:
169             value = INA236_CONVERSION_READY;
170             break;
171         default: return 0;
172     }
173
174     value = value | 0x0000;
175     ina236writeRegister(ina236, INA236_MASK_ENABLE_REGISTER, value);
176     return value;
177 }
178
179 void ina236ResetMaskRegister(Ina236_t *ina236)
180 {
181     ina236writeRegister(ina236, INA236_MASK_ENABLE_REGISTER, 0x0000);
182 }
183
184 void ina236SetCurrentAlertLimit(Ina236_t *ina236, double
185     currentLimit)
186 {
187     double shuntVoltageLimit;
188     uint16_t alertValue;
189
190     shuntVoltageLimit = currentLimit * (ina236->shuntResistor);
191     alertValue = shuntVoltageLimit / (ina236->shuntAdcRange);
192     ina236writeRegister(ina236, INA236_ALERT_LIMIT_REGISTER, alertValue
193         );
194 }
195
196 double ina236ReadShuntVoltage(Ina236_t *ina236)
197 {
198     double result = ina236readRegister(ina236,
199         INA236_SHUNT_VOLTAGE_REGISTER);
200     result = result * (ina236->shuntAdcRange) * (1000.0);

```

```

199     return result;
200 }
201
202 double ina236ReadBusVoltage(Ina236_t *ina236)
203 {
204     double result = ina236readRegister(ina236,
205         INA236_BUS_VOLTAGE_REGISTER);
206     result = result * (1.6) * pow(10, -3);
207     return result;
208 }
209
210 double ina236ReadCurrent(Ina236_t *ina236)
211 {
212     double rawCurrent = ina236readRegister(ina236,
213         INA236_CURRENT_REGISTER);
214     double current = rawCurrent * (ina236->currentLsbMin) * (1000.0);
215     return current;
216 }
217
218 double ina236ReadPower(Ina236_t *ina236)
219 {
220     double rawPower = ina236readRegister(ina236, INA236_POWER_REGISTER)
221     ;
222     double power = 32.0 * rawPower * (ina236->currentLsbMin) * (1000.0)
223     ;
224     return power;
225 }
226
227 _Bool ina236AlertFunctionFlag(Ina236_t *ina236)
228 {
229     uint16_t result = ina236readRegister(ina236,
230         INA236_MASK_ENABLE_REGISTER);
231     _Bool isAlertFunctionFlagTrue = CHECK_BIT(result, 4);
232     return isAlertFunctionFlagTrue;
233 }
234
235 _Bool ina236DataReady(Ina236_t *ina236)
236 {
237     _Bool isDataReady;
238
239     uint16_t conversionReady = ina236readRegister(ina236,
240         INA236_MASK_ENABLE_REGISTER);
241     isDataReady = CHECK_BIT(conversionReady, 3);
242
243     return isDataReady;
244 }

```